Artificial Intelligence

Agents: Laboratory Exercise
Constructing a Goal-Based Agent using a Large Language Model

Learning Objectives

After completing this laboratory you should be able to

1. explain the operation of a goal-based intelligent agent;
2. use a Large Language Model (LLM) to assist with software development;
3. generate, execute and test Python code produced by an LLM;
4. improve software through iterative prompting;
5. critically evaluate the strengths and limitations of LLM-assisted software engineering.

Background

In the accompanying lecture we introduced the notion of a *goal-based agent*.

Unlike a simple reflex agent, a goal-based agent has an explicit objective and selects actions that move it towards that objective.

In this laboratory you will use a Large Language Model (ChatGPT, Gemini, Claude, Copilot or another freely available system) to assist you in constructing such an agent.

The emphasis of this laboratory is not Python programming.

Instead, the emphasis is on

- specifying an AI problem,
- designing an intelligent agent,
- using an LLM as a software engineering assistant,
- testing and validating the generated program.

Software Requirements

You will need

- Python 3.x
- A Python IDE
- Access to a freely available Large Language Model

The Warehouse Navigation Problem

An autonomous warehouse vehicle transports packages from the loading bay to the dispatch area.

The warehouse contains shelving units that cannot be crossed.
The vehicle must determine a collision-free path from its current position to the destination.
The warehouse is represented by the following map.

```
####################
#S....#...........G#
#.##....##########..#
#....##.............#
#.######.###.#.###..#
#........#..........#
####################
```

where

- `S` denotes the starting position,
- `G` denotes the goal,
- `#` denotes an obstacle,
- `.` denotes free space.

The vehicle may move

- Up
- Down
- Left
- Right

One move changes the vehicle's position by one grid square.

Task 1 – Understanding the Problem

Before writing any software, check that you can answer the following questions.

1. What is the environment?
2. What is the goal of the agent?
3. What actions are available to the agent?
4. What information must the agent maintain in order to choose its next action?
5. Why is this an example of a goal-based agent rather than a simple reflex agent?

Think About It

Suppose the warehouse becomes twice as large.
Would the same search strategy still be appropriate?
What additional difficulties might arise?

Task 2 – Designing the Agent

Before asking an LLM to generate code, design the agent.
Your design should identify

- the environment,
- the current state,
- the goal,
- the available actions,
- the decision-making component.

Draw a simple block diagram showing how these components interact.
You may base your design on the goal-based architecture introduced in the lecture.

Task 3 – Prompt Engineering

Use your preferred Large Language Model.
Rather than asking the LLM simply to "write a program", provide it with a clear specification.

For example, you might use a prompt similar to the following.

Suggested Prompt

> Write a well-documented Python program implementing a goal-based agent for the warehouse navigation problem shown above.
>
> The program should
>
> - represent the warehouse as a two-dimensional grid;
> - determine a collision-free path from `S` to `G`;
> - avoid all obstacles;
> - print either the path found or a suitable message if no path exists;
> - explain the search algorithm that has been chosen and why it is appropriate;

TEST TO SEE IF the generated program runs. If not, use the LLM to fix the code. You should finally be able answer the following questions.

1. Did the LLM generate a working program on the first attempt?
2. If not, how can you improve your prompt?
3. What search algorithm did the LLM choose?
4. Why do you think the LLM selected this algorithm?